

Relazione Tecnica

di Progettazione e Sviluppo

Travel Organizer Offline

App mobile per l'organizzazione dei viaggi

Giugno 2026

Gruppo

Componente 1

Componente 2

Componente 3

Indice

1	Descrizione dell'app	2
1.1	Obiettivo	2
1.2	Tipologia di utenti	2
1.3	Problema che l'app intende risolvere	2
1.4	Principali scenari d'uso	2
2	Requisiti	2
2.1	Quadro generale dei requisiti	2
2.2	Modelli dei dati (DF)	2
2.3	Funzionalità implementate (IF)	3
2.4	Feature avanzate scelte	3
2.5	Funzionalità considerate ma non implementate	4
2.6	Limitazioni note	4
3	Progettazione dell'app	4
3.1	Struttura generale	4
3.2	Schermate principali	4
3.3	Flusso di navigazione	4
3.4	Organizzazione dei dati	5
3.5	Integrità dei dati	5
4	Scelte tecnologiche	5
4.1	Framework	5
4.2	Librerie principali	5
4.3	Motivazione delle scelte	6
4.4	Gestione della persistenza	6
5	Implementazione	6
5.1	Organizzazione del codice	6
5.2	Widget principali	6
5.3	Gestione dello stato	6
5.4	Gestione della navigazione	6
5.5	Gestione dei dati persistenti	6
5.6	Parti significative	7
6	Casi d'uso	7
6.1	Elenco dei casi d'uso	7
6.2	Descrizione dettagliata	7
7	Conclusioni	9

1 Descrizione dell'app

1.1 Obiettivo

Travel Organizer Offline è un'applicazione mobile, sviluppata in *Flutter*, che aiuta l'utente a pianificare e gestire i propri viaggi: itinerari, tappe, attività, checklist e spese. Tutte le informazioni restano disponibili **anche senza connessione di rete**, perché i dati sono salvati in un database locale sul dispositivo. L'app non utilizza API remote né un backend: è pensata per funzionare al 100% offline.

1.2 Tipologia di utenti

L'applicazione si rivolge a chiunque organizzi viaggi – viaggiatori abituali, studenti, famiglie – e desideri uno strumento semplice per tenere sotto controllo programma, bagaglio e budget anche dove la rete non è disponibile (aereo, estero, zone remote).

1.3 Problema che l'app intende risolvere

Le informazioni di un viaggio sono spesso frammentate tra note, chat e siti diversi e richiedono una connessione per essere consultate. Travel Organizer Offline centralizza tutto in un'unica app consultabile offline, evitando la dipendenza dalla rete e la dispersione dei dati.

1.4 Principali scenari d'uso

- **Pianificazione:** l'utente crea un viaggio, lo suddivide in tappe e associa a ciascuna le attività con orari e luoghi.
- **Preparazione:** genera la lista valigia in base ai tag del viaggio e compila le checklist dei documenti.
- **Controllo spese:** registra spese previste ed effettive e monitora il budget tramite la barra di salute.
- **In viaggio:** consulta offline la timeline della giornata e i tempi liberi disponibili.
- **Condivisione:** esporta un viaggio in JSON e lo invia a un amico, che può reimportarlo nella propria app.

2 Requisiti

2.1 Quadro generale dei requisiti

2.2 Modelli dei dati (DF)

- **DF-1.1 Viaggio:** id, titolo, destinazione, dataInizio, dataFine, descrizione, stato, budget, partecipanti, note, tag, dataCreazione.
- **DF-1.2 Tappa:** id, viaggioId, titolo, data, località, descrizione, ordine, note.
- **DF-1.3 Attività:** id, viaggioId, tappaId, titolo, descrizione, dataOra, luogo, categoria, costoPrevisto, stato, note.
- **DF-1.4 Checklist:** id, viaggioId, tappaId, titolo, descrizione, elementi.
- **DF-1.5 Elemento checklist:** id, checklistId, titolo, completato, ordine.
- **DF-1.6 Spesa:** id, viaggioId, tappaId, attivitàId, titolo, importo, categoria, data, metodoPagamento, stato (prevista/effettiva), note.

Categoria	Codice	Descrizione	Priorità
Dati	DF-1	Modellazione e persistenza (Viaggi, Tappe, Attività, Checklist, Spese)	Must
Funzione	IF-1	Gestione dei Viaggi	Must
Funzione	IF-2	Gestione delle Tappe/Giornate	Must
Funzione	IF-3	Gestione delle Attività e dell'itinerario	Must
Funzione	IF-4	Gestione delle Checklist e informazioni utili	Must
Funzione	IF-5	Gestione delle Spese e del budget	Must
Funzione	IF-6	Ricerca, filtri e organizzazione	Should
Funzione	IF-7	Riepiloghi e statistiche	Should
Funzione	IF-8	Feature avanzate	Should
Flusso	BF	Flussi inter-modulo	Must

2.3 Funzionalità implementate (IF)

- **IF-1 Viaggi:** elenco, dettaglio, inserimento, modifica, eliminazione, **uplicazione** e assegnazione di **tag**. Lo stato (futuro/in corso/completato/archiviato) è calcolato dalle date.
- **IF-2 Tappe:** elenco, inserimento, modifica, eliminazione, **riordino con drag & drop** e accesso alle attività associate.
- **IF-3 Attività:** inserimento, modifica, eliminazione, associazione a viaggio/tappa, cambio di stato (da fare/completata/annullata), categoria, data/ora, luogo e costo previsto.
- **IF-4 Checklist:** creazione, aggiunta/modifica/eliminazione e spunta degli elementi, barra di avanzamento e duplicazione.
- **IF-5 Spese:** inserimento, modifica, eliminazione, distinzione tra spese previste ed effettive, metodo di pagamento e confronto col budget.
- **IF-6 Ricerca e filtri:** viaggi per titolo/destinazione e per stato; attività per categoria, stato e giorno; spese per stato, categoria e fascia di importo; tappe per nome/località e per data.
- **IF-7 Statistiche:** totali e stato dei viaggi, attività totali e completate, spese effettive/previste, avanzamento checklist, classifica delle tappe con più attività, grafico a torta delle spese per categoria e barra di salute del budget per ciascun viaggio.
- **IF-8 Feature avanzate:** descritte nella Sezione 2.4.

2.4 Feature avanzate scelte

La traccia richiede almeno due feature avanzate; il progetto ne integra diverse, tutte effettivamente funzionanti:

1. **Packing list intelligente con tag.** A ogni viaggio si possono assegnare dei tag (Mare, Montagna, Città, Estero, Business, Trekking, Inverno). Una matrice predefinita nel codice associa a ogni tag un insieme di oggetti (es. *Mare* → Costume, Crema solare; *Estero* → Passaporto, Adattatore prese). La generazione della lista valigia unisce gli oggetti di base con i suggerimenti dei tag, senza duplicati. Tutta la logica è locale.
2. **Timeline visuale con tempi liberi.** L'itinerario è mostrato come linea del tempo con blocchi colorati in base alla categoria; l'app calcola i periodi liberi tra un'attività e l'altra e li evidenzia (es. *"Hai 3h di tempo libero"*).

3. **Budget gamification.** Una barra di salute del budget cambia colore (verde → giallo → rosso) all'aumentare della spesa, e un avviso compare quando una spesa effettiva supera il budget residuo.
4. **Esporta/Importa viaggio.** Un viaggio completo (con tappe, attività, checklist e spese) può essere esportato in JSON e condiviso tramite il menu nativo del sistema; un altro utente può reimportarlo. Gli identificatori vengono rigenerati per evitare conflitti con i dati esistenti.
5. **Duplicazione** di viaggi, tappe e checklist.

2.5 Funzionalità considerate ma non implementate

- Simulatore di cambio valuta con tasso manuale per viaggio.
- Salvataggio di “giornate tipo” come modelli riutilizzabili.
- Temi grafici dinamici in base alla destinazione.
- Notifiche locali (promemoria di partenza/scadenze).

Queste funzioni sono state valutate ma escluse per mantenere l'app semplice, coerente e priva di dipendenze non necessarie.

2.6 Limitazioni note

- La gestione del cambio valuta non è inclusa: gli importi sono in un'unica valuta (Euro).
- L'import di un viaggio avviene incollando il testo JSON (non è prevista la lettura diretta di un file).
- I dati sono locali al dispositivo: non esiste sincronizzazione su cloud.

3 Progettazione dell'app

3.1 Struttura generale

L'app adotta un'architettura a livelli che separa nettamente **interfaccia**, **stato** e **accesso ai dati**:

UI (Schermate) → Provider (Stato) → Repository → Database SQLite

Questa separazione rende il codice ordinato e testabile: le schermate non conoscono il database, ma dialogano con i provider; i provider usano i repository, che incapsulano le query SQL.

3.2 Schermate principali

- **Lista Viaggi:** elenco con ricerca, filtro per stato, tag e azioni rapide (apri, duplica, modifica, elimina); pulsante di importazione.
- **Dettaglio Viaggio:** sei schede – Tappe, Attività, Checklist, Spese, Timeline, Valigia – con esportazione/condivisione.
- **Form** di inserimento/modifica per viaggi, tappe, attività e spese, con validazione dei campi.
- **Dettaglio Tappa:** informazioni della tappa ed elenco attività.
- **Statistiche:** riepiloghi e grafici su tutti i viaggi.

3.3 Flusso di navigazione

La navigazione principale avviene tramite una *Bottom Navigation Bar* a due sezioni (Viaggi e Statistiche). Dalla lista si apre il dettaglio di un viaggio (passaggio dell'id alla schermata successiva), organizzato in schede; da ciascuna scheda si raggiungono i form di inserimen-

to/modifica. Le transizioni usano `Navigator` e `MaterialPageRoute`, con passaggio esplicito dei dati tra schermate.

3.4 Organizzazione dei dati

Lo schema relazionale prevede una tabella per ciascuna entità, con vincoli di chiave esterna e **eliminazione a cascata**: cancellando un viaggio vengono rimossi automaticamente tappe, attività, checklist (con i relativi elementi) e spese collegate.

Tabella	Relazione	Note
trips	–	Entità principale
stages	trips (1:N)	ON DELETE CASCADE
activities	trips/stages	CASCADE su trip, SET NULL su stage
checklists	trips/stages	CASCADE su trip
checklist_items	checklists (1:N)	ON DELETE CASCADE
expenses	trips (1:N)	ON DELETE CASCADE

3.5 Integrità dei dati

Oltre ai vincoli del database, l'app applica regole a livello applicativo:

- impedisce di salvare due viaggi con **date sovrapposte**;
- valida che la data di fine non preceda quella di inizio;
- richiede i campi obbligatori (titolo, destinazione, importo);
- dopo l'eliminazione di un viaggio, allinea le cache in memoria così le statistiche restano coerenti.

4 Scelte tecnologiche

4.1 Framework

È stato scelto **Flutter** (Material 3) per la possibilità di sviluppare con un unico codice sorgente un'app fluida e multiplatforma (Android, iOS, Web), coerentemente con i vincoli della traccia.

4.2 Librerie principali

Libreria	Motivazione
provider	Gestione dello stato semplice e reattiva (pattern <code>ChangeNotifier</code>).
sqflite / sqflite_common_ffi_web	Persistenza locale su SQLite, anche su Web tramite <code>WebAssembly</code> .
fl_chart	Grafici (torta) per la sezione statistiche.
intl	Formattazione di date e valuta in italiano.
share_plus	Condivisione tramite il menu nativo del sistema.
uuid	Generazione di identificatori univoci.

4.3 Motivazione delle scelte

`provider` è leggero e adatto a un'app di queste dimensioni, evitando la complessità di soluzioni più pesanti. `sqflite` fornisce un database relazionale completo (chiavi esterne, cascata) ideale per dati strutturati e collegati. La combinazione con `sqflite_common_ffi_web` permette di eseguire l'app anche nel browser senza modifiche al codice di dominio.

4.4 Gestione della persistenza

Tutti i dati sono salvati in un database **SQLite locale**. Un `DatabaseHelper` (singleton) gestisce apertura, creazione dello schema, attivazione delle chiavi esterne e migrazioni di versione. I `Repository` incapsulano le query SQL (CRUD e ricerche) per ciascuna entità.

5 Implementazione

5.1 Organizzazione del codice

```
lib/  
  core/          Tema, colori, utility, catalogo packing  
  data/  
    database/    DatabaseHelper (schema, migrazioni)  
    models/      Trip, Stage, Activity, Checklist, Expense  
    repositories/ Accesso ai dati (query SQL)  
    trip_transfer.dart Esporta/Importa viaggio (JSON)  
  providers/     Stato dell'app (ChangeNotifier)  
  features/      Schermate (viaggi, tappe, attività, spese, timeline, statistiche)  
  shared/widgets/ Widget riutilizzabili  
  main.dart      Avvio dell'app
```

5.2 Widget principali

Tra i componenti riutilizzabili: `EmptyState` (schermata per le liste vuote con call-to-action), `ConfirmDialog` (conferma delle eliminazioni), `StatusChip` (etichetta colorata di stato). Le schermate di dettaglio usano `TabBar/TabBarView` e `SliverAppBar`.

5.3 Gestione dello stato

Lo stato è gestito con il pattern **Provider/ChangeNotifier**: ogni entità ha un proprio provider (es. `TripProvider`, `ExpenseProvider`) che mantiene i dati in memoria, esegue le operazioni tramite i repository e notifica la UI con `notifyListeners()`. Le schermate si aggiornano automaticamente tramite `Consumer` e `context.watch`.

5.4 Gestione della navigazione

La navigazione tra schermate usa `Navigator` e `MaterialPageRoute`, con **passaggio esplicito dei dati** (ad esempio l'id del viaggio o l'oggetto da modificare). La barra inferiore usa un `IndexedStack` per conservare lo stato delle sezioni.

5.5 Gestione dei dati persistenti

Le operazioni di lettura/scrittura passano sempre dai repository. Esempio di controllo di integrità sulle date dei viaggi:

```
// Due intervalli si sovrappongono se: inizio <= altraFine && altroInizio <= fine
Trip? overlappingTrip(DateTime start, DateTime end, {String? excludeId}) {
  for (final t in _trips) {
    if (t.id == excludeId) continue;
    if (!start.isAfter(t.endDate) && !t.startDate.isAfter(end)) return t;
  }
  return null;
}
```

5.6 Parti significative

- **Packing list intelligente:** un catalogo tag → oggetti guida la generazione automatica della valigia.
- **Esporta/Importa:** serializzazione dell'intero grafo di un viaggio in JSON e re-importazione con rimappatura degli identificatori, validata da un test automatico di round-trip.
- **Timeline:** calcolo dei tempi liberi tra attività consecutive dello stesso giorno.
- **Test:** il progetto include test automatici (avvio dell'app, integrità delle date, eliminazione, esporta/importa) eseguiti su SQLite via FFI.

6 Casi d'uso

6.1 Elenco dei casi d'uso

ID	Caso d'uso	ID	Caso d'uso
UC-1	Inserisci viaggio	UC-7	Cerca e filtra elementi
UC-2	Inserisci tappa	UC-8	Visualizza statistiche
UC-3	Pianifica attività	UC-9	Visualizza timeline
UC-4	Gestisci checklist	UC-10	Esporta / Importa viaggio
UC-5	Registra spesa	UC-11	Modifica o elimina elemento
UC-6	Genera packing list smart		

6.2 Descrizione dettagliata

UC-1: Inserisci viaggio

Attore: Utente.

Precondizioni: App avviata, persistenza inizializzata.

Postcondizioni: Un nuovo viaggio è salvato e mostrato nell'elenco.

Flusso base:

1. L'utente seleziona "Nuovo viaggio".
2. Compila titolo, destinazione, date, budget, partecipanti, tag e note.
3. Il sistema valida i campi obbligatori e le date.
4. Il sistema salva il viaggio e torna all'elenco aggiornato.

Flussi alternativi:

- **3a. Date sovrapposte:** il sistema rileva un altro viaggio con date che si accavallano, blocca il salvataggio e mostra un avviso.
- **3b. Campi non validi:** il sistema evidenzia l'errore e resta in attesa di una correzione.

UC-2: Inserisci tappa

Attore: Utente.

Precondizioni: Esiste un viaggio.

Postcondizioni: La tappa è aggiunta all'itinerario del viaggio.

Flusso base: l'utente apre la scheda Tappe, inserisce titolo, data, località, descrizione e note; il sistema valida e salva, aggiornando l'elenco (riordinabile tramite drag & drop).

UC-3: Pianifica attività

Attore: Utente.

Precondizioni: Esiste un viaggio (ed eventualmente una tappa).

Postcondizioni: L'attività è pianificata e associata al viaggio/tappa.

Flusso base: l'utente inserisce titolo, categoria, data/ora, luogo, costo e tappa associata; il sistema valida e salva. **Alternativo:** titolo mancante → il salvataggio è bloccato con messaggio d'errore.

UC-4: Gestisci checklist

Attore: Utente.

Postcondizioni: La checklist e i suoi elementi sono aggiornati.

Flusso base: l'utente crea una checklist, aggiunge elementi, li spunta o li elimina; la barra di avanzamento si aggiorna in tempo reale. Può inoltre duplicare una checklist esistente.

UC-5: Registra spesa

Attore: Utente.

Postcondizioni: La spesa è registrata e i totali aggiornati.

Flusso base: l'utente inserisce titolo, importo, categoria, data, tipo (prevista/effettiva) e metodo di pagamento; il sistema valida e salva.

Alternativo 3a. Fuori budget: se la spesa effettiva supera il budget residuo, il sistema mostra un avviso e chiede conferma prima di salvare.

UC-6: Genera packing list smart

Attore: Utente.

Precondizioni: Esiste un viaggio (preferibilmente con tag).

Postcondizioni: La lista valigia è popolata con oggetti pertinenti.

Flusso base:

1. L'utente apre la scheda Valigia e seleziona "Genera lista".
2. Il sistema legge i tag del viaggio e recupera dal catalogo gli oggetti suggeriti.
3. Il sistema unisce oggetti di base e suggerimenti dai tag, senza duplicati, e li aggiunge alla checklist.

UC-7: Cerca e filtra elementi

Attore: Utente.

Flusso base: l'utente applica ricerca o filtri (viaggi per titolo/stato; attività per categoria/stato/giorno; spese per categoria/importo; tappe per nome/data) e la lista mostra solo i risultati pertinenti.

Alternativo: nessun risultato → viene mostrato uno stato vuoto esplicativo.

UC-8: Visualizza statistiche

Attore: Utente.

Flusso base: l'utente apre la sezione Statistiche; il sistema calcola totali, attività completate, spese e avanzamento checklist, e genera i grafici e le barre di salute del budget.

Alternativo: archivio vuoto → messaggio “Nessun dato disponibile”.

UC-9: Visualizza timeline

Attore: Utente.

Flusso base: l'utente apre la scheda Timeline; il sistema mostra le attività come blocchi colorati per categoria, ordinate per orario, ed evidenzia i tempi liberi tra un'attività e l'altra.

UC-10: Esporta / Importa viaggio

Attore: Utente.

Postcondizioni: Un viaggio è condiviso o ricreato nell'archivio.

Flusso base (Esporta): dal dettaglio del viaggio l'utente sceglie “Esporta”; il sistema genera il JSON e apre il menu di condivisione nativo.

Flusso base (Importa): dall'elenco l'utente incolla il JSON; il sistema lo valida, rigenera gli identificatori, ricollega le relazioni e crea il nuovo viaggio.

Alternativo: JSON non valido → messaggio d'errore.

UC-11: Modifica o elimina elemento

Attore: Utente.

Flusso base: l'utente seleziona “Modifica” o “Elimina”; in caso di eliminazione il sistema chiede conferma con un `AlertDialog`, poi aggiorna il database e la UI.

Alternativi: annullamento della rimozione (nessuna modifica) oppure dati modificati non validi (salvataggio bloccato con segnalazione).

7 Conclusioni

Travel Organizer Offline soddisfa tutte le funzionalità richieste dalla traccia – viaggi, tappe, attività, checklist e spese – aggiungendo ricerca, filtri, statistiche e diverse feature avanzate effettivamente integrate. L'architettura a livelli, la persistenza locale coerente e i controlli di integrità rendono l'app robusta, semplice da usare e completamente funzionante offline.